CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 4: Design Multi-Agent Systems and Orchestration

Exam objective: Design multi-agent systems using coordinator and subagent architectures, including task delegation and decomposition, choices between parallel and sequential execution, context passing between agents, clearly scoped agent responsibilities and tool access, management of token consumption, concurrency and execution limits, and handling of failed or incomplete subagent runs.

Q1 **An architect is designing a multi-agent research system for an investment firm. A coordinator agent delegates company-specific research to multiple subagents. What is the PRIMARY risk if each subagent is given unrestricted access to every internal tool, including trade-execution APIs?**

A. There is no risk; broad access always improves subagent flexibility

B. Subagents may inadvertently invoke sensitive or irreversible actions (such as executing trades) outside the scope of their research task, violating least-privilege principles

C. Broad tool access reduces token consumption

D. Broad tool access eliminates the need for a coordinator

CORRECT ANSWER **B**

WHY Each subagent should be scoped to the minimum tools necessary for its task; granting access to irreversible, high-impact tools like trade execution for a research task violates least-privilege and creates real operational risk. A and C are incorrect generalizations, and D confuses tool access with coordination responsibility.

Q2 **A coordinator agent in a healthcare claims system delegates three independent subtasks: verifying patient eligibility, checking provider network status, and validating procedure codes. What is the BEST execution strategy?**

A Run all three subtasks sequentially to simplify debugging regardless of dependency

B Run the three subtasks in parallel, since each is independent and their results can be combined once all complete

C Run only one subtask and infer the other two

D Run them in a random order each time to spread load

CORRECT ANSWER B

WHY Because the three subtasks have no dependencies on one another, running them in parallel minimizes total latency while preserving correctness, which is the core benefit of recognizing independence during decomposition. Sequential execution here only adds unnecessary latency, and the other options sacrifice correctness or introduce needless unpredictability.

Q3 **A multi-agent customer-service system has a coordinator that passes the full conversation transcript, including irrelevant small talk, to every subagent. What problem does this design create?**

A It improves subagent accuracy with no downside

B It unnecessarily increases token consumption and may dilute each subagent's focus with irrelevant context, increasing cost and risk of distraction

C It eliminates the need for context passing entirely

D It guarantees subagents will fail

CORRECT ANSWER B

WHY Passing unfiltered, bloated context to every subagent wastes tokens and can distract a model from the specific subtask at hand; context passed between agents should be curated to what is relevant. A overstates the benefit, C mischaracterizes the described design, and D is an overstatement of the consequence.

Q4 **In an orchestrator-workers architecture for a manufacturing quality system, one worker subagent times out while analyzing a batch of sensor readings. What is the BEST way for the coordinator to handle this?**

A Silently discard the entire batch's results, including the subagents that succeeded

B **Treat the timeout as a partial failure: retry or flag the specific failed worker's subtask, while retaining the results of the subagents that completed successfully**

C Halt the entire system until an engineer manually intervenes

D Have the coordinator fabricate a plausible result for the failed worker

CORRECT ANSWER B

WHY A resilient multi-agent design isolates failures to the specific failed unit of work, allowing successful results to be retained and only the failed portion retried or escalated, rather than discarding good work or fabricating data. A wastes valid results, C is an overreaction that harms availability, and D introduces dangerous fabricated data into a quality system.

Q5 **A legal tech company builds a multi-agent contract review system. Which design BEST reflects clear agent responsibility boundaries?**

A. A single agent that both drafts contract language and also makes the final approval decision with no separation

B. One subagent scoped to clause extraction only, a second subagent scoped to risk-flagging only, and a coordinator that assembles their outputs into a unified review, with a human making the final approval

C. All subagents sharing one combined prompt with no distinct roles

D. A coordinator that performs all the actual analysis itself while subagents remain idle

CORRECT ANSWER B

WHY Clearly scoping each subagent to a distinct, testable responsibility (extraction versus risk-flagging) with the coordinator assembling results, and a human retaining final approval authority, exemplifies sound separation of concerns in multi-agent design. A collapses drafting and approval inappropriately, C and D both fail to establish meaningful role boundaries.

Q6 **An e-commerce platform's multi-agent system experiences runaway cost because a research subagent repeatedly re-queries the same product database without any limit. What architectural control is MOST important to add?**

A. A larger context window for the subagent

B. An explicit execution limit (maximum tool calls, iterations or token budget) per subagent, with the coordinator monitoring and halting subagents that exceed it

C. Removing the subagent's tool access entirely

D. Switching to a sequential-only architecture

CORRECT ANSWER B

WHY Bounding token consumption, iteration counts and tool-call limits per subagent, enforced by coordinator-level monitoring, is the standard control against runaway cost in agentic systems. A larger context window does not address runaway looping, removing tool access eliminates needed functionality, and switching to sequential execution does not address the root cause of unbounded iteration.

Q7 **A telecom company's coordinator agent delegates network diagnostics to five subagents running concurrently. The architect must decide on a concurrency limit. What is the BEST basis for this decision?**

A. An arbitrary round number with no analysis

B. The downstream API rate limits, available compute budget, and the acceptable cost/latency trade-off for the business use case

C. The number of engineers on the team

D. The subagents' prompt lengths only

CORRECT ANSWER B

WHY Concurrency limits should be grounded in real constraints — downstream system rate limits, compute/cost budget, and business-acceptable latency — rather than arbitrary choices. C and D are irrelevant factors to this specific technical decision.

Q8 **Which scenario BEST justifies using a multi-agent system rather than a single augmented LLM call?**

A Answering a single customer's account balance question

B Conducting a comprehensive competitive market analysis requiring independent deep research across many distinct competitors, industries and data sources, later synthesized into one report

C Converting a single PDF to plain text

D Classifying a single support ticket's urgency

CORRECT ANSWER **B**

WHY Multi-agent orchestration earns its added complexity when a task naturally decomposes into multiple independent, substantial research threads that benefit from separate context and parallel execution before synthesis. The other scenarios are simple, single-step tasks that do not justify the overhead of multiple agents.

Q9 **A financial audit firm's multi-agent system passes a subagent's full raw output, including internal reasoning and scratch notes, back to the coordinator, which then forwards it unfiltered to the client-facing report. What risk does this illustrate?**

A No risk; more detail is always better for clients

B Internal reasoning or exploratory notes not intended for external audiences may leak into client deliverables, and context passed between agents should be filtered to what is appropriate for the next consumer

C The subagent used too few tokens

D The coordinator should be eliminated since it adds no value

CORRECT ANSWER **B**

WHY Context handoffs between agents and, ultimately, to external outputs must be deliberately filtered to the appropriate audience and purpose; raw internal reasoning is not automatically safe or suitable for client-facing content. A is a poor practice, C is irrelevant, and D ignores the coordinator's legitimate assembly role.

Q10 **A government agency's orchestrator delegates document redaction verification to three subagents, each checking a different exemption category. One subagent returns a malformed, unparseable response. What is the BEST design response?**

A Treat the entire multi-agent run as successful and ignore the malformed response

B **Have the coordinator detect the malformed response through schema/output validation, retry that specific subagent's task, and escalate to a human if retries continue to fail**

C Automatically terminate the other two subagents' valid results as well

D Permanently disable that subagent's task category going forward

CORRECT ANSWER B

WHY Robust orchestration requires validating subagent outputs and isolating failure handling (retry, then escalate) to the specific failed component without discarding other valid results, which supports both reliability and auditability. A risks silent compliance failure, C destroys valid work unnecessarily, and D is a disproportionate permanent response to a transient failure.

Q11 An architect is deciding between sequential and parallel execution for a coordinator managing subagents in an insurance underwriting pipeline, where subagent B's risk score depends on subagent A's completed fraud check. What is the correct design?

A Run A and B in parallel since multi-agent systems should always parallelize

B Run A first, then B sequentially, because B has a genuine data dependency on A's output

C Run B first to save time, then backfill A's result afterward

D Merge A and B into a single subagent regardless of their distinct responsibilities

CORRECT ANSWER B

WHY Parallel execution is only valid for independent subtasks; here B's risk score explicitly depends on A's fraud-check output, so sequential execution respecting the dependency is required for correctness. A misapplies a general parallelization preference despite an explicit dependency, C violates the dependency order, and D conflates distinct responsibilities that should remain separately scoped and testable.

Q12 **A manufacturing company's multi-agent system coordinator must decide how much context to pass to a newly spawned subagent tasked with inspecting a single production line's sensor logs. What is the BEST practice?**

A Pass the entire company-wide sensor history across all production lines for maximum context

B **Pass only the context relevant to that specific production line and task, keeping the subagent's scope and token usage tightly bounded**

C Pass no context at all and let the subagent guess

D Pass the coordinator's own internal planning notes verbatim

CORRECT ANSWER **B**

WHY Scoping context to exactly what a subagent needs for its specific delegated task keeps token consumption efficient and the subagent's focus sharp, which is a core principle of context passing in multi-agent design. A wastes tokens and risks irrelevant distraction, C deprives the subagent of necessary information, and D leaks coordinator-internal detail with no clear benefit.

Q13 **A retail chain's multi-agent inventory reconciliation system occasionally produces inconsistent final numbers because two subagents independently update the same shared inventory record without coordination. What is the BEST architectural fix?**

A Allow it to continue since inconsistencies are rare

B Ensure the coordinator serializes or locks writes to shared state, or restructures the subtask boundaries so subagents never write to the same resource concurrently

C Increase the number of subagents to catch more errors

D Remove the coordinator so subagents can self-organize freely

CORRECT ANSWER B

WHY Concurrent, uncoordinated writes to shared state are a classic race-condition risk in multi-agent systems; the fix is either serialized access to shared resources or a task decomposition that avoids overlapping writes. A accepts a correctness bug, C increases the chance of further collisions, and D removes the very coordination needed to prevent this issue.

Q14 **Which of the following is the STRONGEST indicator that a task should be handled by a single agent rather than a multi-agent system?**

A The task requires reading one short document and answering one question about it with no independent subtasks

B The task requires independently researching ten unrelated topics

C The task requires coordinating distinct specialized roles across a long-running investigation

D The task has subtasks with no dependencies that can run concurrently

CORRECT ANSWER A

WHY A single, self-contained task with no natural decomposition into independent subtasks does not benefit from the coordination overhead of a multi-agent system; a single agent call is simpler, cheaper and equally effective. B, C and D all describe genuine decomposition opportunities that justify multi-agent coordination.

Q15 **A SaaS company's coordinator agent delegates five subtasks to subagents and must synthesize their outputs into one final answer. Two subagents return results, but three fail due to an upstream API outage. What is the BEST coordinator behavior?**

A Return a final answer as if all five subtasks had succeeded

B **Recognize the partial failure, clearly communicate which parts of the analysis are incomplete, and either retry the failed subtasks or deliver a transparently partial result with the gap identified**

C Silently cache the two successful results forever and never retry

D Abandon the entire task and return no output at all

CORRECT ANSWER B

WHY Transparent handling of partial failures — communicating exactly what succeeded and what did not, with a retry or clearly flagged gap — preserves trust and usability even under degraded conditions. A fabricates completeness dishonestly, C provides no path to recovery, and D discards legitimate partial value unnecessarily.

Q16 **A company is considering splitting its single customer-service agent into three separate department-specific agents (billing, technical support, retention) purely because the organization has three separate teams that each want ownership of "their own" agent. The task itself — answering a customer's question — has no contradictory instructions or tool-access requirements across departments, and tickets are not independent of each other (the same conversation often spans all three topics). What is the BEST assessment?**

A. The split is justified because team ownership and organizational structure are sufficient reasons to create separate agents

B. The split is not justified by the task itself; only irreconcilable specialization (contradictory instructions or tool access) or genuine independence between subtasks justify a second agent, and organizational boundaries like team ownership are facts about the company, not evidence that one agent is insufficient

C. The split is justified because three agents will always be more accurate than one

D. The split is justified because more agents always reduce latency

CORRECT ANSWER B

WHY Only two properties of the work itself justify splitting into multiple agents: instructions or tool access that genuinely cannot coexist, or sub-tasks that are truly independent of one another; team structure, headcount and departmental ownership describe the organization rather than the task and do not establish that a single agent falls short, especially when conversations span all three topics and thus are not independent. A, C and D all offer reasons unrelated to a genuine architectural requirement.

Q17 **A coordinator in a retail-operations system sends three tool calls to subagents in a single turn: a price-check, an inventory-check, and a promotion-eligibility check. The coordinator decides not to execute the promotion-eligibility check because the promotions service is under maintenance. What is the coordinator REQUIRED to do with that declined call?**

A Omit any result for the declined call, since it was never actually executed

B **Still return a result for the declined call in the next message, matched to its call identifier and marked as an error with a brief explanation, alongside the results for the two calls that did run**

C Wait indefinitely until the promotions service comes back online before responding at all

D Fabricate a plausible promotion-eligibility result so the response looks complete

CORRECT ANSWER **B**

WHY When multiple tool calls are issued in one turn, every call must receive a matched result in the next message, including calls that were deliberately not run, which must still be returned as an error result with a brief explanation; silence is not a valid response to any call. A leaves an unresolved call the system cannot reconcile, C stalls the entire turn over one declined call, and D introduces fabricated data that misrepresents an unavailable check as a completed one.

Q18 **A logistics company's multi-agent system imposes a per-subagent task budget intended to cap how many tokens a research subagent can consume during a long investigation. The architect is told this budget must act as an absolute, unbreakable ceiling that can never be exceeded under any circumstance. What should the architect point out?**

A Task budgets are exactly the right mechanism for an absolute, unbreakable ceiling

B **A task budget is a soft hint that paces the model but that the model may exceed when stopping mid-action would be more disruptive than finishing; if an absolute hard limit is required, it must be enforced through the request's maximum token setting instead**

C There is no way to limit token consumption in a multi-agent system

D Task budgets and maximum token settings are two names for the same mechanism with identical enforcement guarantees

CORRECT ANSWER **B**

WHY A task budget is documented as a soft, self-paced allowance that the model may exceed rather than a hard enforcement mechanism, so a genuine hard ceiling on spend must come from the request's maximum token setting, which is the actual enforced limit. A misstates the budget's guarantee, C ignores a real control that does exist (just not the one asked for), and D collapses a meaningful distinction between a soft pacing hint and a hard cap. --